

GEMV 5: Multiple PEs

Contents

- [layout.csl](#)
- [pe_program.csl](#)
- [run.py](#)
- [commands.sh](#)

Continuing on from the previous example, we now extend our program to use multiple PEs.

The number of PEs used in this program is set at compile-time with the `width` parameter. Note that `layout.csl` uses this parameter to set the size of the program with the call to `@set_rectangle`. The dimensions of a grid of PEs is always specified as width by height (or, alternatively, number of columns by number of rows), and individual PEs are indexed by (x, y), or, in other words, (column number, row number).

This program involves no communication between PEs; we only duplicate the same workload on each PE. In `run.py`, the `memcpy_h2d` calls now specify that data is copied into `width x 1` PEs, beginning at the upper left corner (0, 0) of the program rectangle. Because we are copying the same data to each PE, we use `np.tile` to repeat the data in `A`, `x`, and `b` multiple times. The `memcpy_d2h` call copies back the resulting `y` from each PE into an array of size `M x width`.

The next example will expand this example to demonstrate simple communication between PEs.

Note

See [GEMV Tutorial 5: Multiple PEs](#) for a step-by-step walkthrough of this example.

layout.csl

```

// matrix dimensions on each PE
param M: i16;
param N: i16;

// number of PEs in program
param width: i16;

const memcpy = @import_module("<memcpy/get_params>", .{
    .width = width,
    .height = 1
});

layout {
    // PE coordinates are (column, row)
    @set_rectangle(width, 1);
    for (@range(i16, width)) |x| {
        @set_tile_code(x, 0, "pe_program.csl", .{
            .memcpy_params = memcpy.get_params(x),
            .M = M,
            .N = N
        });
    }

    // export symbol names
    @export_name("A", [*]f32, true);
    @export_name("x", [*]f32, true);
    @export_name("b", [*]f32, true);
    @export_name("y", [*]f32, false);
    @export_name("compute", fn()void);
}

```

pe_program.csl

```

param memcpy_params;

// Matrix dimensions
param M: i16;
param N: i16;

// memcpy module provides infrastructure for copying data
// and launching functions from the host
const sys_mod = @import_module("<memcpy/memcpy>", memcpy_params);

// 48 kB of global memory contain A, x, b, y
var A: [M*N]f32; // A is stored row major
var x: [N]f32;
var b: [M]f32;
var y = @zeros([M]f32); // Initialize y to zero

// DSDs for accessing A, b, y

```

```

// A_dsd accesses column of A
var A_dsd = @get_dsd(mem1d_dsd, .{ .tensor_access = |i|{M} -> A[i*N] });
var b_dsd = @get_dsd(mem1d_dsd, .{ .base_address = &b, .extent = M });
var y_dsd = @get_dsd(mem1d_dsd, .{ .base_address = &y, .extent = M });

// ptrs to A, x, b, y will be advertised as symbols to host
var A_ptr: [*]f32 = &A;
var x_ptr: [*]f32 = &x;
var b_ptr: [*]f32 = &b;
const y_ptr: [*]f32 = &y;

// Compute gemv
fn gemv() void {
    // Loop over all columns of A
    for (@range(i16, N)) |i| {
        // Calculate contribution to A*x from ith column of A, ith elem of x
        @fmacs(y_dsd, y_dsd, A_dsd, x[i]);
        // Move A_dsd to next column of A
        A_dsd = @increment_dsd_offset(A_dsd, 1, f32);
    }
    // Add b to A*x
    @fadds(y_dsd, y_dsd, b_dsd);
}

// Call initialize and gemv functions
fn compute() void {
    gemv();
    sys_mod.unblock_cmd_stream();
}

comptime {
    @export_symbol(A_ptr, "A");
    @export_symbol(x_ptr, "x");
    @export_symbol(b_ptr, "b");
    @export_symbol(y_ptr, "y");
    @export_symbol(compute);
}

```

run.py

```

#!/usr/bin/env cs_python

import argparse
import json
import numpy as np

from cerebras.sdk.runtime.sdkruntimepybind import SdkRuntime, MemcpyDataType, MemcpyOrder #
pylint: disable=no-name-in-module

# Read arguments
parser = argparse.ArgumentParser()
parser.add_argument('--name', help="the test compile output dir")

```

```

parser.add_argument('--cmaddr', help="IP:port for CS system")
args = parser.parse_args()

# Get matrix dimensions from compile metadata
with open(f"{args.name}/out.json", encoding='utf-8') as json_file:
    compile_data = json.load(json_file)

# Matrix dimensions
N = int(compile_data['params']['N'])
M = int(compile_data['params']['M'])

# Number of PEs in program
width = int(compile_data['params']['width'])

# Construct A, x, b
A = np.arange(M*N, dtype=np.float32)
x = np.full(shape=N, fill_value=1.0, dtype=np.float32)
b = np.full(shape=M, fill_value=2.0, dtype=np.float32)

# Calculate expected y
y_expected = A.reshape(M,N)@x + b

# Construct a runner using SdkRuntime
runner = SdkRuntime(args.name, cmaddr=args.cmaddr)

# Get symbols for A, x, b, y on device
A_symbol = runner.get_id('A')
x_symbol = runner.get_id('x')
b_symbol = runner.get_id('b')
y_symbol = runner.get_id('y')

# Load and run the program
runner.load()
runner.run()

# Copy A, x, b to device
runner.memcpy_h2d(A_symbol, np.tile(A, width), 0, 0, width, 1, M*N, streaming=False,
    order=MemcpyOrder.ROW_MAJOR, data_type=MemcpyDataType.MEMCPY_32BIT, nonblock=False)
runner.memcpy_h2d(x_symbol, np.tile(x, width), 0, 0, width, 1, N, streaming=False,
    order=MemcpyOrder.ROW_MAJOR, data_type=MemcpyDataType.MEMCPY_32BIT, nonblock=False)
runner.memcpy_h2d(b_symbol, np.tile(b, width), 0, 0, width, 1, M, streaming=False,
    order=MemcpyOrder.ROW_MAJOR, data_type=MemcpyDataType.MEMCPY_32BIT, nonblock=False)

# Launch the init_and_compute function on device
runner.launch('compute', nonblock=False)

# Copy y back from device
y_result = np.zeros([M*width], dtype=np.float32)
runner.memcpy_d2h(y_result, y_symbol, 0, 0, width, 1, M, streaming=False,
    order=MemcpyOrder.ROW_MAJOR, data_type=MemcpyDataType.MEMCPY_32BIT, nonblock=False)

# Stop the program
runner.stop()

# Ensure that the result matches our expectation

```

```
np.testing.assert_allclose(y_result, np.tile(y_expected, width), atol=0.01, rtol=0)
print("SUCCESS!")
```

commands.sh

```
#!/usr/bin/env bash
```

```
set -e
```

```
cslc --arch=wse3 ./layout.csl --fabric-dims=11,3 \  
--fabric-offsets=4,1 --params=M:4,N:6,width:4 -o out --memcpy --channels 1  
cs_python run.py --name out
```

© Copyright 2026, Cerebras Systems.

Last updated on Apr 15, 2026, 7:13:14 PM.